



**UNIVERSIDAD DE GUADALAJARA**  
**CUCEI**



**Carrera: Ingeniería en Computación**  
**Materia: Arquitectura de Computadoras**

**AVANCE**  
**IMPLEMENTACIÓN MÓDULO SPI**



**Integrantes:**

**Hernández Zúñiga Roberto      López López Karla Valeria**  
**Ortiz Segura Emmanuel Alejandro      Segura Gutierrez Aaron Ricardo**

**Maestro: López Arce Delgado Jorge Ernesto**

**Sección: D03**

**Fecha: 13 marzo 2026**

# Índice

- 1. Roles y Responsabilidades**
- 2. Avance Técnico (Verilog y lógica)**
  - 2.1 Definición de módulos
  - 2.2 Diagrama de FSM
  - 2.3 Plan de Pruebas (Test Bench)
- 3. Avance de Reporte Técnico**
  - 3.1 Introducción
  - 3.2 Objetivos
  - 3.3 Marco Teórico
    - 3.3.1 ¿Cómo funciona SPI?
    - 3.3.2 Modos de operación de SPI
    - 3.3.3 Estados de conexión
- 4. Avance de video (Tik Tok/Short)**
  - 4.1 Guión Literario
  - 4.2 Planificación Visual
- 5. Avance de Presentación**
  - 5.1 Estructura de Diapositivas
  - 5.2 Diagrama de Bloques del Sistema
- 6. Bibliografía**

# 1. Roles y Responsabilidades

## **Integrantes**

- Hernández Zúñiga Roberto

### **Avance de video**

- Guión literario
- Planificación visual

- López López Karla Valeria

### **Avance Técnico** (Verilog y lógica)

- Definición de módulos
- Plan de Pruebas
- Diagrama de FSM y de Bloques del Sistema

- Ortiz Segura Emmanuel Alejandro

### **Avance de Presentación**

- Estructura de diapositivas
- Parte de la bibliografía

- Segura Gutierrez Aaron Ricardo

### **Avance de Reporte Técnico**

- Introducción, marco teórico y objetivos
- Bibliografía

## 2. Avance Técnico (Verilog y lógica)

### 2.1 Definición de Módulos

#### Módulos (maestro, esclavo y SPI)

```
////modulo maestro
module Master(
    input clk,
    input rst,
    input start,
    input [7:0] tx_data,
    input miso,
    output reg mosi,
    output reg sclk,
    output reg ss
);

//estadossss
localparam IDLE = 2'b00, START = 2'b01, SHIFT = 2'b10, STOP = 2'b11;

reg [1:0] state;
reg [7:0] shift_reg;
reg [3:0] bit_count;
reg [3:0] clk_cnt;

//falta

endmodule

//modulo esclavo
module slave(
    input clk,
    input sclk,
    input smosi,
    input sss,
    output reg smiso
);

reg [7:0] rx_reg;
reg sclk_old;

//apartado por agregar

endmodule

//////////topmodule spi
module SPI_top (
```

```

    input CLK,
    input RST,
    input START,
    input [7:0] DATA
);

wire c1, c2, c3, c4;

Master m1 (
    .clk(CLK),
    .rst(RST),
    .start(START),
    .tx_data(DATA),
    .miso(c4),
    .mosi(c1),
    .sclk(c2),
    .ss(c3)
);

slave s1 (
    .clk(CLK),
    .sclk(c2),
    .smosi(c1),
    .sss(c3),
    .smiso(c4)
);

endmodule

```

## Código en capturas

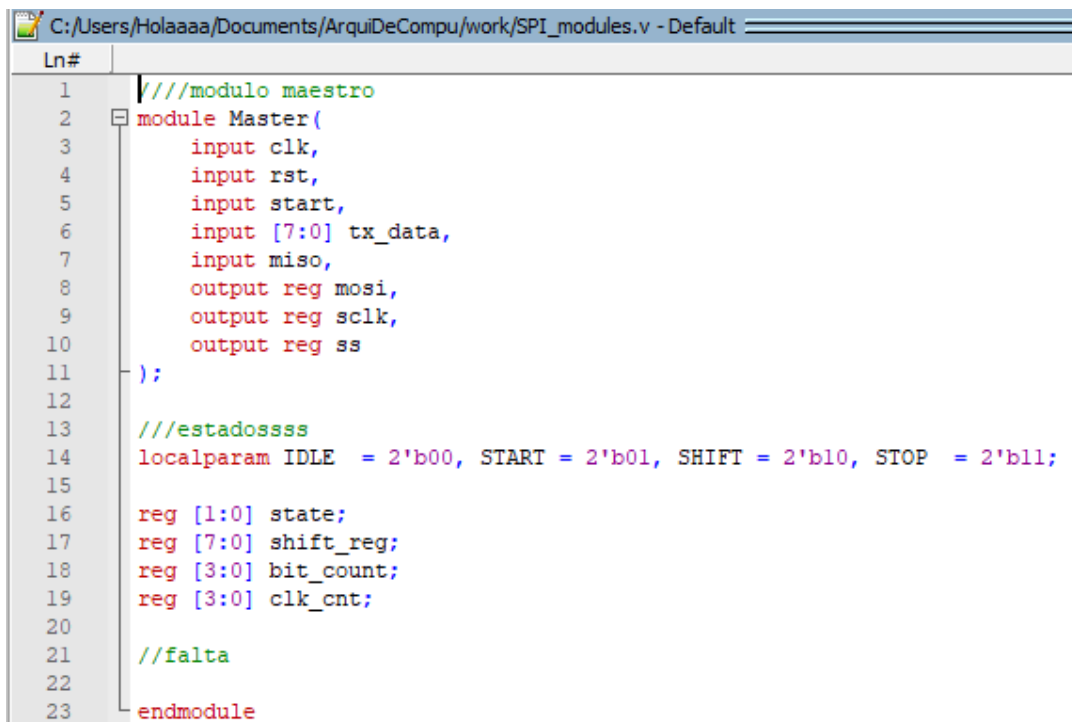
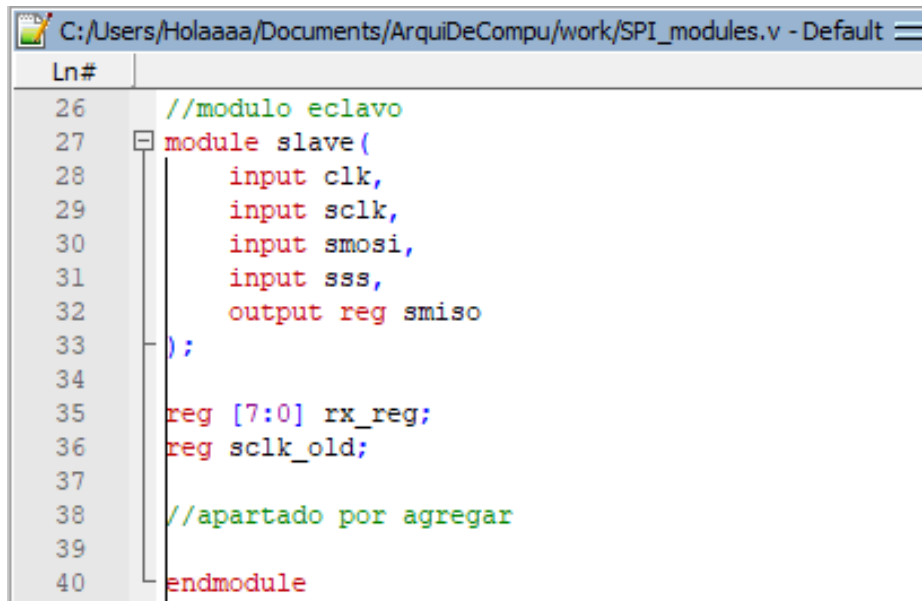
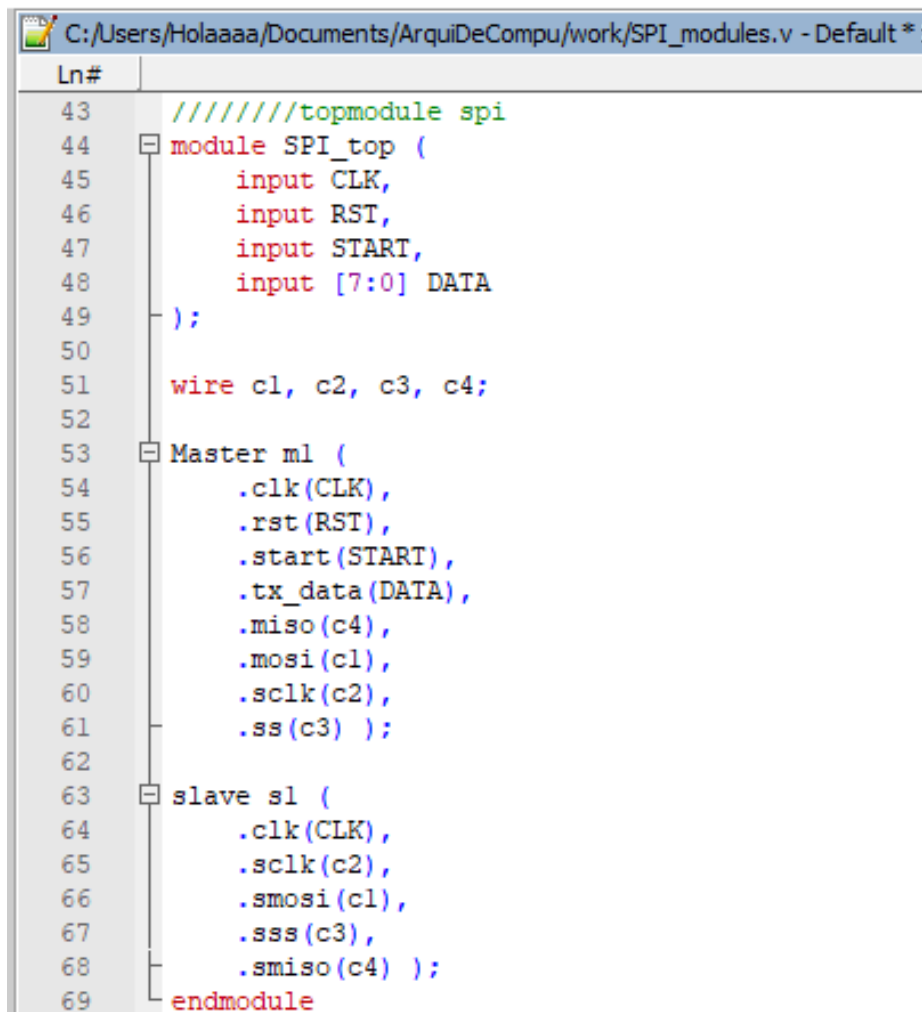


Imagen del módulo Master en ModelSim.



```
C:/Users/Holaaaa/Documents/ArquiDeCompu/work/SPI_modules.v - Default
Ln#
26 //modulo esclavo
27 module slave(
28     input clk,
29     input sclk,
30     input smosi,
31     input sss,
32     output reg smiso
33 );
34
35 reg [7:0] rx_reg;
36 reg sclk_old;
37
38 //apartado por agregar
39
40 endmodule
```

Imagen del módulo Slave en ModelSim.

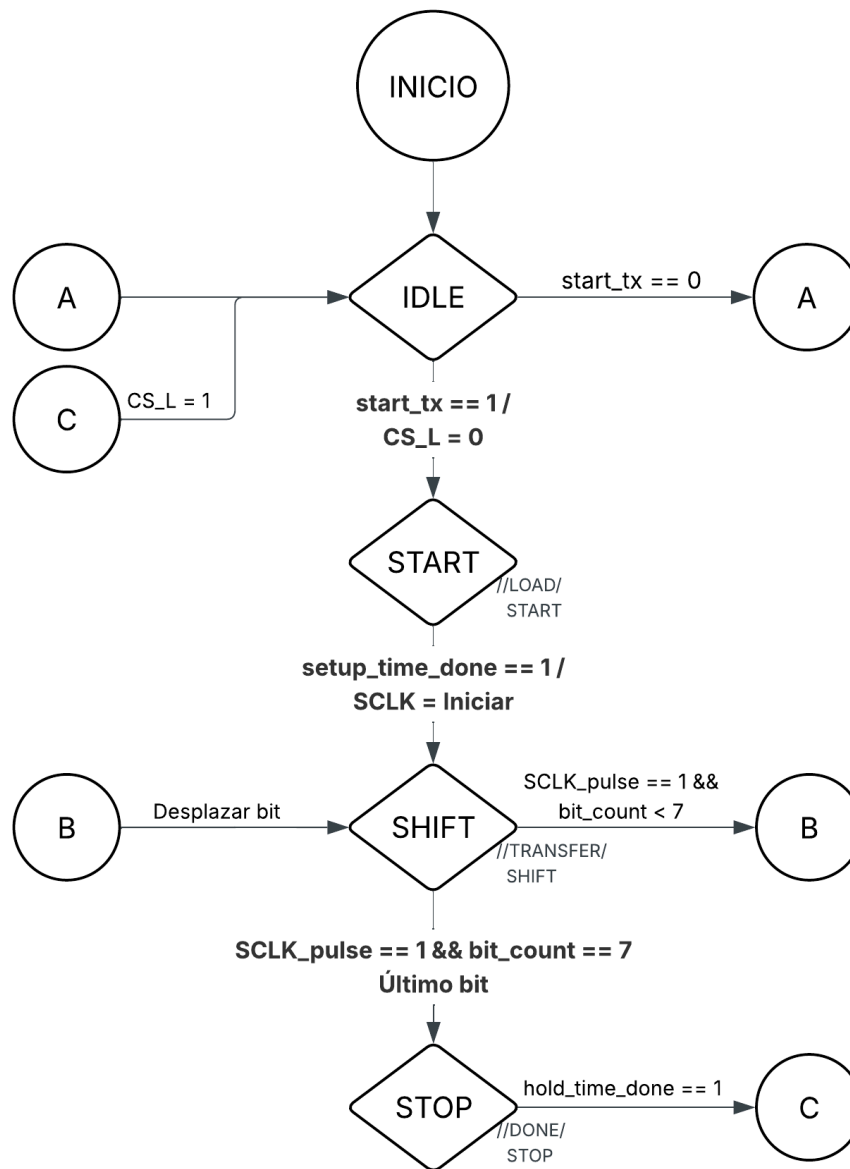


```
C:/Users/Holaaaa/Documents/ArquiDeCompu/work/SPI_modules.v - Default *
Ln#
43 //topmodule spi
44 module SPI_top (
45     input CLK,
46     input RST,
47     input START,
48     input [7:0] DATA
49 );
50
51 wire c1, c2, c3, c4;
52
53 Master m1 (
54     .clk(CLK),
55     .rst(RST),
56     .start(START),
57     .tx_data(DATA),
58     .miso(c4),
59     .mosi(c1),
60     .sclk(c2),
61     .ss(c3) );
62
63 slave s1 (
64     .clk(CLK),
65     .sclk(c2),
66     .smosi(c1),
67     .sss(c3),
68     .smiso(c4) );
69 endmodule
```

Imagen del módulo SPI\_top (top module) en ModelSim.

## 2.2 Diagrama de FSM

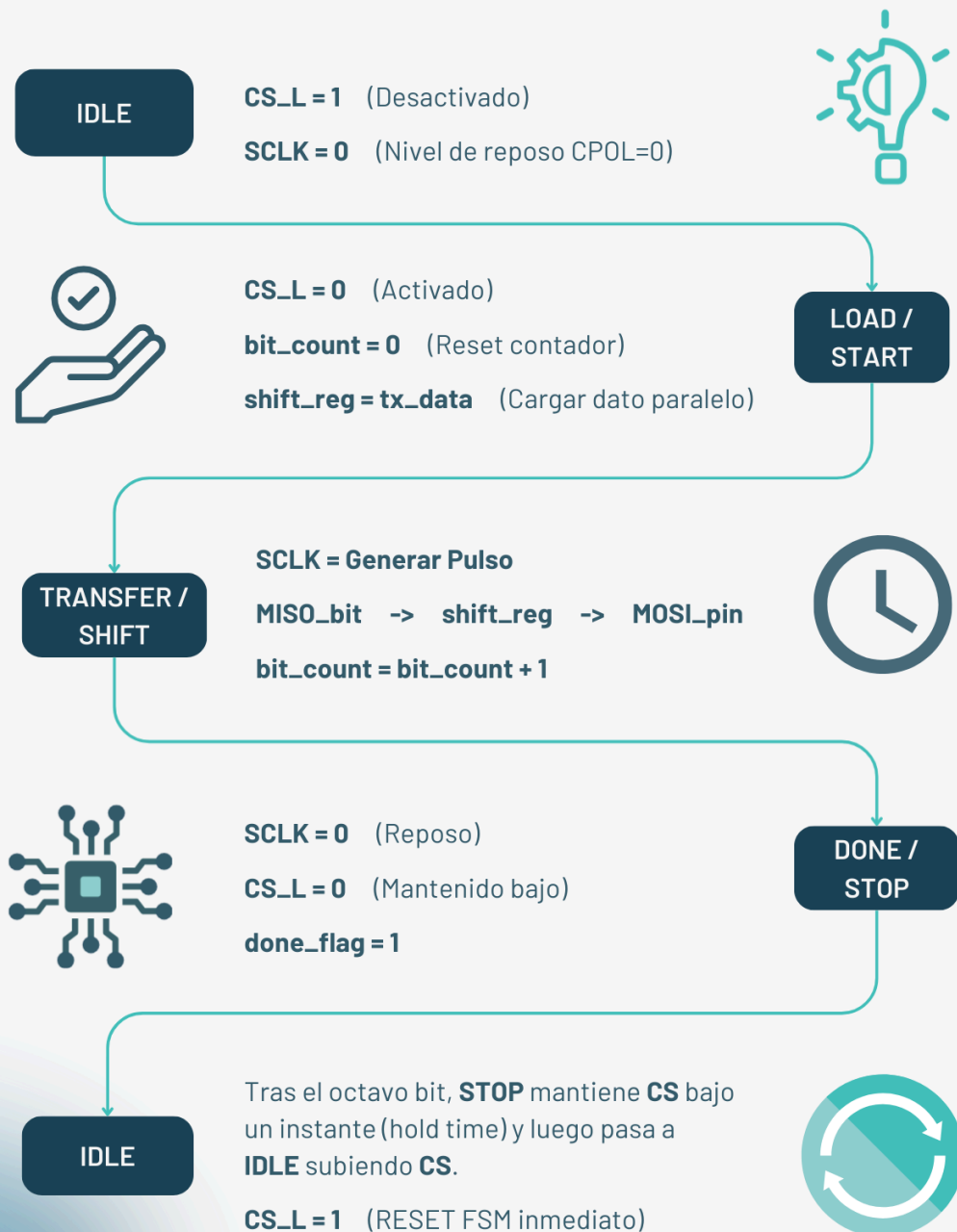
Diagrama Máquina de Estados Finitos (FSM)



## Desglose Diagrama FSM

# Desglose Diagrama FSM

### Acciones de Control (Salidas)





## 2.3 Plan de Pruebas (Test Bench)

### Caso 1.

#### Validación de Tiempos de Control (CS y setup time)

Este caso busca verificar que la FSM no inicie la transmisión de forma abrupta, respetando los estados de preparación.

- **Estímulo:** Activar la señal **start\_tx** con un valor de datos fijo (ej. **0xAA** (8 bits)).
- **Resultado esperado:**
  1. La señal **Chip Select (CS)** debe bajar a nivel lógico '0'.
  2. Debe observarse un retardo (correspondiente al estado **START**) antes del primer flanco de **SCLK**.
  3. Verificar que el estado de la FSM cambie de **IDLE** a **START** y luego a **SHIFT**.

### Caso 2.

#### Integridad de Transmisión de Byte Completo

Se verifica que el Contador de Bits y el Registro de Desplazamiento trabajen en sincronía para enviar exactamente 8 bits.

- **Estímulo:** Cargar el dato **0xFF** (todos los bits en alto) y posteriormente **0x00**.
- **Resultado esperado:**
  1. Contar exactamente 8 pulsos de la señal de reloj **SCLK**.
  2. Observar en la señal **MOSI** la salida serial bit a bit.
  3. La FSM debe permanecer en el estado **SHIFT** durante los 8 pulsos y transicionar a **STOP** inmediatamente después del último flanco.

### Caso 3.

#### Verificación de Fin de Trama y Retorno a Reposo (hold time)

Este caso asegura que el sistema se limpie correctamente y quede listo para una nueva transmisión, evitando colisiones de datos.

- **Estímulo:** Finalizar la transmisión del octavo bit.
- **Resultado esperado:**
  1. La señal **SCLK** debe detenerse y quedar en su nivel de reposo (según el modo CPOL elegido).
  2. La señal **CS** debe permanecer en '0' durante un breve periodo (estado **STOP/DONE**) antes de subir a '1'.
  3. La bandera busy o done debe actualizarse correctamente para indicar al sistema que el bus está libre.

### ¿Qué vamos a probar?

| Señal         | Origen      | Objetivo                                              |
|---------------|-------------|-------------------------------------------------------|
| current_state | FSM Interna | Validar que recorre IDLE - START - SHIFT -STOP        |
| SCLK          | Maestro     | Verificar frecuencia y número de pulsos (8)           |
| MOSI / MISO   | Bus Serial  | Comprobar que el dato enviado coincide con el cargado |

## 3. Avance de Reporte Técnico

### 3.1 Introducción

Los protocolos de comunicación son necesarios en el ámbito de la computación, pues estos definen la forma en la que los diferentes componentes en un sistema deben interactuar entre ellos para lograr el resultado esperado. Los protocolos de comunicación, como su nombre lo indica, siguen una serie de reglas que especifican cuándo y cómo deben de reaccionar y realizar acciones los componentes dentro del protocolo. Es importante aclarar que los protocolos de comunicación no reaccionan a la información recibida ni interactúan con ella, sólo se encargan de trasladarla.

En la década de los 80 surge un problema con los microcontroladores, antes de SPI, conectar un microcontrolador a diversos periféricos era una tarea complicada, cada uno requería de una serie de muchos cables (buses paralelos con múltiples líneas de datos y control), lo que terminaba haciendo que los circuitos necesarios fueran extremadamente complejos. Una de las mayores necesidades era reducir la cantidad de conexiones, cables necesarios y que a su vez no se perdiera la velocidad de transferencia de datos, incluso con la posibilidad de aumentarlos.

En este punto es donde nos encontramos otro problema, a la hora de realizar más conexiones físicas, el microcontrolador no era capaz de enviar y recibir todos los datos de manera ordenada, y llegaba un punto donde simplemente no lograba entender qué era lo que recibía y por lo tanto, no saber cómo reaccionar o realizar acciones no esperadas.

Entendido esto, podemos resumir el motivo del desarrollo de SPI en tres puntos principales:

- Reducir el número de conexiones físicas
- Incrementar la velocidad de transmisión de datos
- Permitir la conexión de múltiples dispositivos

SPI nace con este planteamiento en mente, un sistema de comunicación en serie (necesario para utilizar una sola o mínimas líneas de comunicación) y que a su vez tuvieran un orden y tiempo en el que se deben de enviar los datos (un reloj).

Durante este documento estaremos desarrollando el concepto de comunicación dentro de la computación, el de SPI y sobre este último concepto desarrollaremos la parte teórica y técnica del protocolo.

A su vez, veremos para qué se utiliza dentro de la industria, ejemplos de aplicaciones del protocolo, ventajas que utilizarlo ofrece, así como los problemas que pudieran presentarse al utilizarlo y, de manera breve, explicar alternativas y sus principales diferencias.

## 3.2 Objetivos

### Objetivo general

Explicar, paso por paso, el funcionamiento de SPI, su origen, la necesidad que motivó su desarrollo y su utilidad en sistemas electrónicos, documentando la información más importante.

### Objetivos específicos

- Dejar claro el funcionamiento de SPI
- Dar ejemplos de su uso en la industria
- Realizar un ejercicio práctico del uso de SPI
- Obtener la información para una presentación que recopile la información más importante
- Investigar lo necesario para realizar un video sobre SPI de forma didáctica y sencilla

## 3.3 Marco Teórico

SPI, es un protocolo desarrollado por Motorola a mediados de 1980 para proveer un lenguaje en el que sea posible la comunicación entre los dispositivos electrónicos, la comunicación suele ocurrir en distancias cortas, como las utilizadas en una placa de circuitos o dispositivos electrónicos pequeños, naturalmente es utilizada cuando existe una relativamente baja cantidad de transmisión de datos y la aplicación más común que tiene SPI es la comunicación entre un microcontrolador y un sensor, como la activación de una luz con sensor de movimiento.

SPI forma parte de los protocolos de comunicación los cuales tienen como objetivo llevar la información de punto A al punto B, para esto, utilizamos canales de comunicación que conectan estos puntos A y B.

En dichos puntos se encuentran los equipos transmisores y receptores de datos, así como codificadores y decodificadores que interpretan la información para que ambos dispositivos sean capaces de entender las señales enviadas.

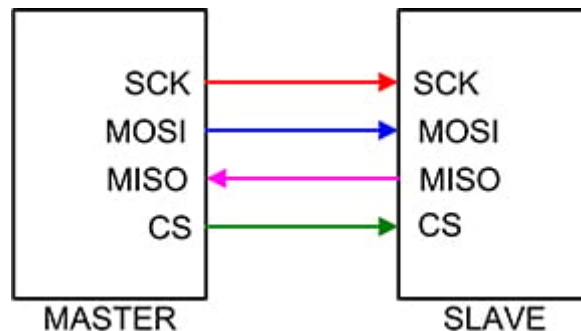
El acrónimo SPI en programación e ingeniería electrónica significa Serial Peripheral Interface (Interfaz Periférica Serial). Es un protocolo de comunicación síncrono, maestro-esclavo, utilizado para la transferencia rápida de datos a corta distancia entre microcontroladores y periféricos (sensores, memorias).

El motivo de la popularidad de SPI radica en sus muchas ventajas. La primera es que es una interfaz direccionada de hardware simple que ofrece completa flexibilidad para la cantidad de bits transferidos. Usa un modelo de maestro-esclavo con un maestro simple y puede manejar varios dispositivos secundarios (esclavos) usando comunicaciones dúplex (permite el intercambio de datos bidireccional entre dos dispositivos, con modo Full-Duplex y Half-Duplex), que operan a velocidades de

reloj de hasta 50 MHz. No usa un protocolo estándar y transfiere solo paquetes de datos, lo que la hace ideal para transferir flujos de datos largos.

### 3.3.1 ¿Cómo funciona SPI?

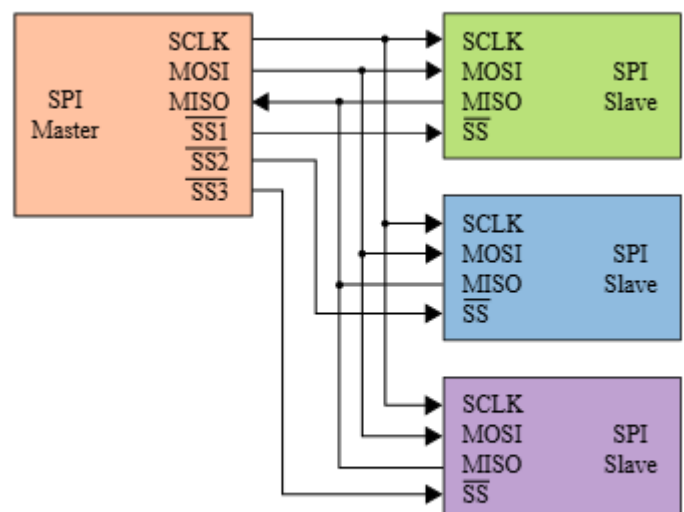
La comunicación SPI funciona con el modelo maestro-esclavo, donde el dispositivo maestro controla la señal de reloj (SCLK) e inicia las transferencias de datos. El protocolo utiliza cuatro líneas:



- **MOSI** (Master Out Slave In): Transporta datos del maestro al esclavo.
- **MISO** (Maestro En Esclavo Fuera): Transporta datos del esclavo al maestro.
- **SCLK** (reloj serie): Sincroniza la transmisión de datos.
- **SS** (selección de esclavo): Selecciona el dispositivo esclavo con el que comunicarse.

Los datos se transfieren simultáneamente en las líneas MOSI y MISO, sincronizados por los pulsos de reloj generados por el maestro. Esta comunicación full-duplex permite un intercambio de datos eficiente y continuo.

Una de las principales cualidades de SPI es que el master puede tener varios slaves, agregando un pin adicional por cada slave que se desee agregar.



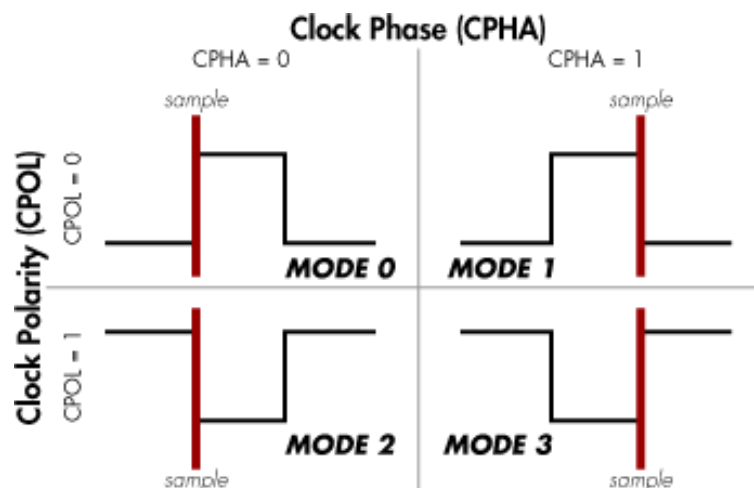
### 3.3.2 Modos de operación de SPI

El comportamiento de SPI depende de los parámetros CPOL (Clock Polarity) y CPHA (Clock Phase).

- **Clock Polarity** determina el nivel lógico base del reloj cuando el bus se encuentra inactivo, si CPOL = 0, el reloj permanece en nivel bajo, si CPOL = 1, el reloj permanece en nivel alto, ambos durante el estado inactivo, este parámetro define en qué borde del reloj se muestrean los datos, esto no cambia el orden de los datos, sólo define en qué nivel se encuentra el reloj a la hora de comenzar el ciclo.
- **Clock Phase** determina en qué borde del reloj se toman los datos. Si CPHA = 0, los datos se toman en el primer borde del reloj después de que se inicia el ciclo (ascendente o descendente dependiendo de CPOL). Si CPHA = 1, se toman en el segundo borde del reloj, a diferencia de CPOL que define el nivel base del reloj, CPHA define el momento exacto en el que se leen o escriben los bits.

Combinando CPOL y CPHA obtenemos los cuatro modos de operación de SPI:

- Modo 0: CPOL=0, CPHA=0
- Modo 1: CPOL=0, CPHA=1
- Modo 2: CPOL=1, CPHA=0
- Modo 3: CPOL=1, CPHA=1



De esta forma SPI se adapta a las necesidades de los diferentes dispositivos que lo requieran, asegurando que maestro y esclavo interpreten correctamente los datos.

Los estados mínimos o configuraciones fundamentales de SPI se definen principalmente por la combinación de la polaridad del reloj (CPOL) y la fase del reloj

(CPHA), lo que resulta en 4 modos de operación distintos. Además, de un conjunto mínimo de líneas físicas para funcionar.

### 3.3.3 Estados de Conexión

- **Idle (Reposo/Inactivo):** Cuando SS está en alto, el bus no está transmitiendo, y SCLK está en bajo (Modo 0/1) o alto (Modo 2/3).
- **Transmisión (Activo):** Cuando SS pasa a bajo, el maestro genera el reloj y los datos se intercambian simultáneamente en ambas direcciones (dúplex completo).

La velocidad de transmisión (frecuencia) no tiene un estándar único y puede ajustarse dividiendo el reloj del controlador (ej. 16 MHz en Arduino).

## 4. Avance de Video (Tik Tok/Short)

### 4.1 Guión Literario

#### Introducción

“Si alguna vez te preguntaste cómo hablan los sensores con un microcontrolador, hoy te explico uno de los métodos más usados: SPI

#### ¿Qué es SPI?

“SPI significa Serial Peripheral Interface.

Básicamente es una forma en la que los dispositivos electrónicos se mandan datos entre ellos.”

“Es como un chat, pero entre chips.”

#### ¿Dónde se usa?

“Se usa en muchísimas cosas como:

- sensores
- pantallas
- memorias
- y varios dispositivos electrónicos.”

“Si hay un microcontrolador... probablemente hay SPI por ahí.”

#### ¿Cómo funciona?

“SPI funciona con un jefe y los que obedecen. El maestro controla todo y los esclavos solo responden cuando el maestro les habla.”

“Para comunicarse usan cuatro cables importantes:

- MOSI: el maestro manda datos
- MISO: el esclavo responde
- SCLK: el reloj que marca el ritmo
- CS: para elegir con qué dispositivo hablar”

#### Estados mínimos

“Un módulo SPI normalmente pasa por estados como:

1. Esperar
2. Cargar datos
3. Enviar los bits
4. Terminar la transmisión”

“Y luego vuelve a empezar.”

#### Formato de trama

“Los datos se mandan bit por bit, siguiendo el ritmo del reloj.”

“Muchas veces se envían 8 bits por mensaje, aunque puede variar.”

Cierre

“Y listo. Así funciona SPI: chips hablando entre ellos súper rápido.”

## 4.2 Planificación Visual

### Escena 1 (Hook)

Visual

Texto grande:

“Cómo hablan los chips entre ellos”

Animación simple de microcontrolador y sensor.

### Escena 2 (Qué es SPI)

Visual

Diagrama simple:

Microcontrolador - dispositivo

Texto:

“SPI = comunicación entre dispositivos”

### Escena 3 (Dónde se usa)

Visual

Íconos de:

- sensor
- pantalla
- memoria
- muchos dispositivos más que llenen la pantalla

Texto:

“Se usa en MUCHOS dispositivos”

### Escena 4 (Cómo funciona)

Visual

Diagrama maestro-esclavo.

Texto en pantalla:

MOSI → datos

MISO ← respuesta

SCLK → reloj

CS → selección

### Escena 5 (Estados)

Visual

Diagrama simple:

Esperar → Cargar → Enviar → Terminar

### Escena 6 (Trama)

Visual



Animación:

SCLK: ----\_

MOSI: 10110110

Texto:

“Bits enviados con el reloj”

## **Escena 7 (Cierre)**

Visual

Texto grande:

“SPI = comunicación rápida entre chips”

## 5. Avance de la Presentación

### 5.1 Estructura de Diapositivas

#### Índice (Presentación)

1. Introducción  
¿Qué es y para qué sirve?
2. Conexiones  
¿Cómo se realizan las conexiones internas?
3. Máquina de estados  
¿Cómo se controla la comunicación?
4. Modos del SPI  
¿Cómo se sincronizan los datos CPOL y CPHA?
5. Ejemplos  
¿Dónde se ve esto y cómo se implementa?
6. Conclusiones  
Importancia de conocer el protocolo

### 5.2 Diagrama de Bloques del Sistema

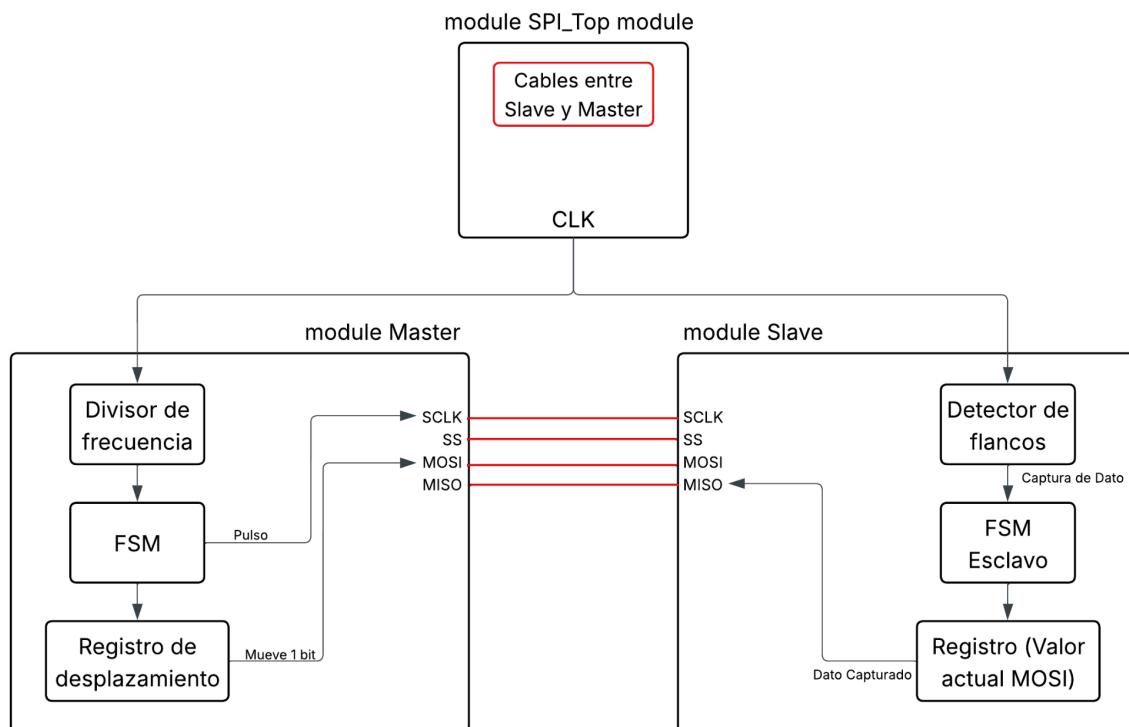


Diagrama de bloques para el sistema (módulos).

## 6. Bibliografía

- Panox Display. (2025, 10 de julio). ¿Qué es la comunicación SPI y cómo funciona? <https://www.panoxdisplay.com/es/solution/what-is-spi-communication-and-how-does-it-work/>
- Digi-Key Electronics. (2019, 14 de febrero). SPI simplifica la comunicación con dispositivos: Por qué y cómo usar Serial Peripheral Interface para simplificar las conexiones entre múltiples dispositivos. <https://www.digikey.es/es/articles/why-how-to-use-serial-peripheral-interface-simplify-connections-between-multiple-devices>
- Texas Instruments. (2025). Understanding the SPI Bus (SBOA621). <https://www.ti.com/lit/ab/sboa621/sboa621.pdf>
- Interfaz periférica en serie (SPI) - SparkFun Learn. (n.d.). <https://learn.sparkfun.com/tutorials/serial-peripheral-interface-spi/all>
- Paraguay. (2026, January 9). Protocolo SPI (Serial Peripheral Interface). MCI Educación. <https://cursos.mcielectronics.cl/2022/08/23/serial-peripheral-interface-spi/>
- How are CPOL, CPHA and Bitrate Configured for a Host Adapter that is in SPI Slave Mode? (2026, March 4). Total Phase Blog. [https://www.totalphase.com/blog/2014/12/cpol-cpha-bitrate-configured-host-adapter-spi-slave-mode/?srsltid=AfmBOqrkeglvUduj1NAYLYfQZ\\_w3h8Jc-ilBwJuZ5NGG-J-eJ0kDSUO](https://www.totalphase.com/blog/2014/12/cpol-cpha-bitrate-configured-host-adapter-spi-slave-mode/?srsltid=AfmBOqrkeglvUduj1NAYLYfQZ_w3h8Jc-ilBwJuZ5NGG-J-eJ0kDSUO)
- Figure 2. Four working mode timing diagram Mode 0: CPOL = 0, CPHA=0. ... (n.d.). ResearchGate. [https://www.researchgate.net/figure/Four-working-mode-timing-diagram-Mode-0-CPOL-0-CPHA0-The-SCK-serial-clock-line-idle\\_fig2\\_339088398](https://www.researchgate.net/figure/Four-working-mode-timing-diagram-Mode-0-CPOL-0-CPHA0-The-SCK-serial-clock-line-idle_fig2_339088398)
- Microtecnología. (2022, August 18). Conceptos básicos comunicación SPI [Video]. YouTube. <https://www.youtube.com/watch?v=CeVmpxBEpXs>
- Wels. (2018, March 12). #1MinutoDeElectronica - Que es SPI - Wels Theory [Video]. YouTube. <https://www.youtube.com/watch?v=FzP65txocIM>
- IBM Business Automation Workflow. (2026, January 29). <https://www.ibm.com/docs/es/baw/23.0.x?topic=types-state-machines>
- What are clock signals in digital circuits, and how are they produced? (n.d.). Symmetry Electronics. <https://www.symmetryelectronics.com/blog/what-are-clock-signals-in-digital-circuits-and-how-are-they-produced-symmetry-blog/>
- TheBestPlayer. (2022, September 13). Understanding SPI CPOL and CPHA. Stack Overflow. <https://stackoverflow.com/questions/73699021/understanding-spi-cpol-and-cpha>
- Getting Started with STM32 - How to Use SPI. (n.d.). DigiKey. <https://www.digikey.com/en/maker/projects/getting-started-with-stm32-how-to-use-spi/09eab3dfe74c4d0391aaaa99b0a8ee17>